

Advanced computerarchitecture
Exercise 4 Report

(B-KUL-JPI30R)

Wout Van Steenbergen

March 17, 2024



Session Date: March 4 & 11, 2024

Instructor: Alex Marinšek

1 Introduction

In this report, two distinct concept are explained. The first focuses on the implementation of a kernel to calculate the average grayscale of an image, examining the differences in execution time between coalesced and un-coalesced memory access. The second part delves into matrix multiplication, assessing how the use of different memory types (global, shared, and constant) affects processing time. Each exercise aims to highlight the importance of memory access patterns and their impact on performance within NVIDIA GPUs.

2 Coalesced and uncoalesced memory access

In CUDA programming, memory access patterns significantly impact the performance of GPU-based computations. Coalesced memory access occurs when threads of a warp access consecutive memory addresses, allowing the GPU to combine these accesses into a single transaction. This leads to more efficient memory usage and higher performance. The principle of coalesced memory access leverages DRAM's capability to access data in blocks and grouping access. Un-coalesced memory access, on the other hand, happens when the threads access memory in a scattered manner, causing multiple memory transactions. This results in slower performance due to the increased load on the memory bandwidth. To illustrate the performance difference between both patterns, 1D arrays that stores an image are converted to grayscale.

- **Un-coalesced:** The image is stored in an array with RGB values sequentially as RGBRGB...RGB. This arrangement allows for coalesced memory access because consecutive threads can read consecutive memory locations when processing each pixel's RGB values.
- **Coalesced:** The image is stored in three separate blocks: RR...R GG...G BB...B. This arrangement leads to un-coalesced memory access because processing a single pixel's grayscale value requires accessing widely separated memory locations.

Two kernels are designed, each optimized for one of the storage arrangements. The first kernel (for the un-coalesced pattern) must jump across memory segments to gather the RGB components for each pixel, while the second kernel (for the coalesced pattern) accesses memory sequentially. (correction of prior, both actually do coalesced memory access)

3 Global, shared, and constant memory

CUDA offers different types of memory, each with its own characteristics and usage recommendations:

- **Global memory:** Resides in the device's DRAM and is accessible by all threads across all blocks. It has the largest storage capacity but also the highest latency and is not cached. Ideal for storing large datasets but due to its high access latency, optimizing access patterns (like ensuring coalesced access) are crucial.
- **Shared Memory:** Much faster than global memory and accessible only by threads within the same block. It allows for efficient inter-thread communication, reducing global memory accesses and improving performance. Its size is limited.
- **Constant Memory:** Cached and fast, but read-only and of limited size. Best used for data that does not change over the course of a kernel execution and is shared across all threads like constants or lookup tables.

3.1 Matrix multiplication

To illustrate the differences in performance between using global memory, shared memory, and constant memory, a common task like matrix multiplication is used.

Global Memory Kernel: This kernel performs matrix multiplication using global memory for both input matrices (A and B) and the output matrix (C). Each thread calculates one element of the result matrix by determining its row and column in the output matrix. It then iterates over a row of A and a column of B (Figure 1).

Constant Memory Kernel: Similar in structure to the global memory kernel, this variant utilizes two constant memory arrays (cd_A and cd_B) for the input matrices (Figure 1).

Shared Memory Kernel: This advanced kernel divides the computation into blocks and uses shared memory within each block to store sub-matrices of A and B. Each thread block computes a sub-matrix of the result matrix C by loading blocks of A and B into shared memory. This method requires synchronization points to ensure all threads have completed their computations before moving on to the next set of sub-matrices (Figure 2).

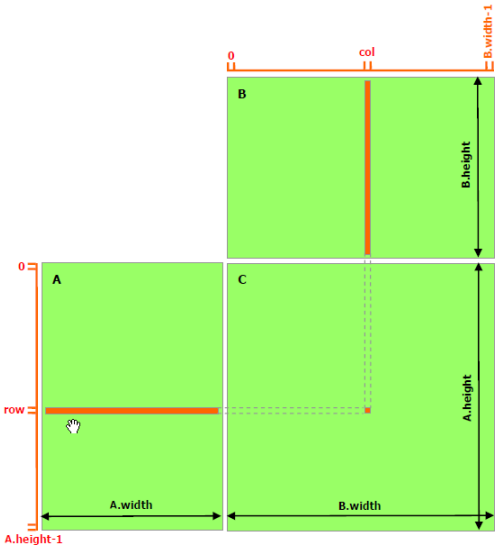


Figure 1: Matrix multiplication without shared memory [1].

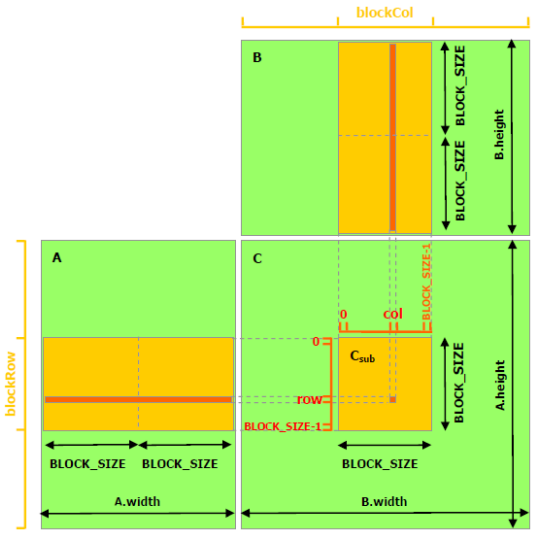


Figure 2: Matrix multiplication with shared memory [1].

4 Results

The first part focuses on the different in processing time for coalesced vs un-coalesced memory access, by converting an image to grayscale using two distinct memory access patterns: an RGBRGB...RGB arrangement and a reorganized, format RR...RGG...GBB...B.

Figure 3 shows the processing time between the two for in increasing number of threads and one block. Initially, the kernel processing the reorganized (coalesced) array shows the best performance compared to its un-coalesced counterpart. This performance can be attributed to the memory access pattern in the coalesced arrangement, which benefits from increased cache utilization when access is sequential within a single color channel. The structure of the reorganized array aligns with the access pattern of threads, leading to improved cache hits and reduced global memory latency.

However, as we scaled up the grid size to 1024 threads, the performance dynamics shifted dramatically. This can be seen in Figure 4. The kernel operating on the RGBRGB...RGB array show in sudden decrease in processing time while the reformatted version increased drastically. This shift is probably due to the limitations of cache memory and the overhead associated with managing a larger number of threads. It's also possible that the first format aligns well with the GPU's internal optimizations and that the opposite is true for the other format.

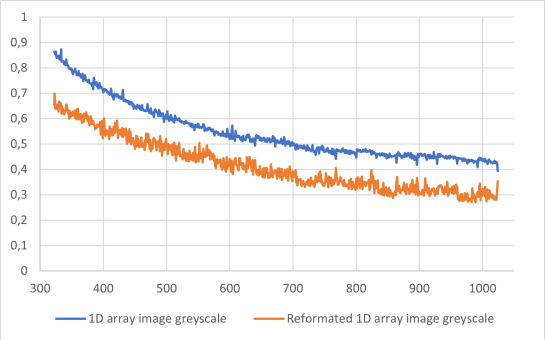


Figure 3: processing time (ms) of kernels for n number of threads.

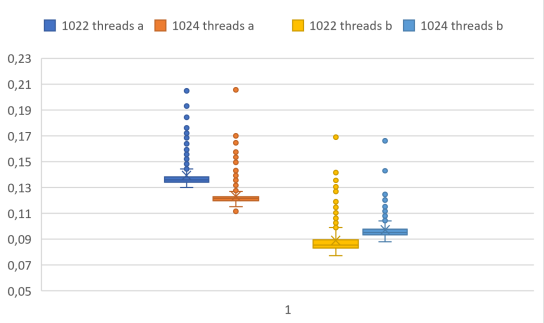


Figure 4: average processing time (ms) (1k runs) a: normal format, b: reformated format.

The second part focuses on the different in processing time for the different memory types, by using matrix multiplications. A number of matrix sizes and grid sizes where tested, but only the result of 88x88 matrix, which is the largest number for which two of them can be plugged into constant memory, is shown. the BLOCK_SIZE is 22 which is the largest divisor of 88 that is smaller than 32.

Shared memory yields the fastest execution times for the matrix multiplication operation (see Figure 5). This is expected since shared memory is much faster than global memory due to its proximity to the processing units and its higher bandwidth. It is designed to be used by threads within the same block, enabling efficient inter-thread communication and fast access to data that is frequently used or shared among threads. The execution times for global memory are slower than for shared memory, which aligns with the understanding that global memory has higher latency and lower bandwidth compared to shared memory.

Surprisingly, constant memory shows the slowest execution times. This result may seem counter intuitive at first, as constant memory is typically cached and provides fast read access for all threads. However, constant memory is intended for scenarios where all threads are reading the same unchanging data. In the context of matrix multiplication, where each thread may require different elements from the matrices, the benefits of constant memory caching are not realized. Additionally, if the constant memory is not large enough to contain the entire matrix or if there are bank conflicts due to improper access patterns, it can significantly degrade performance.

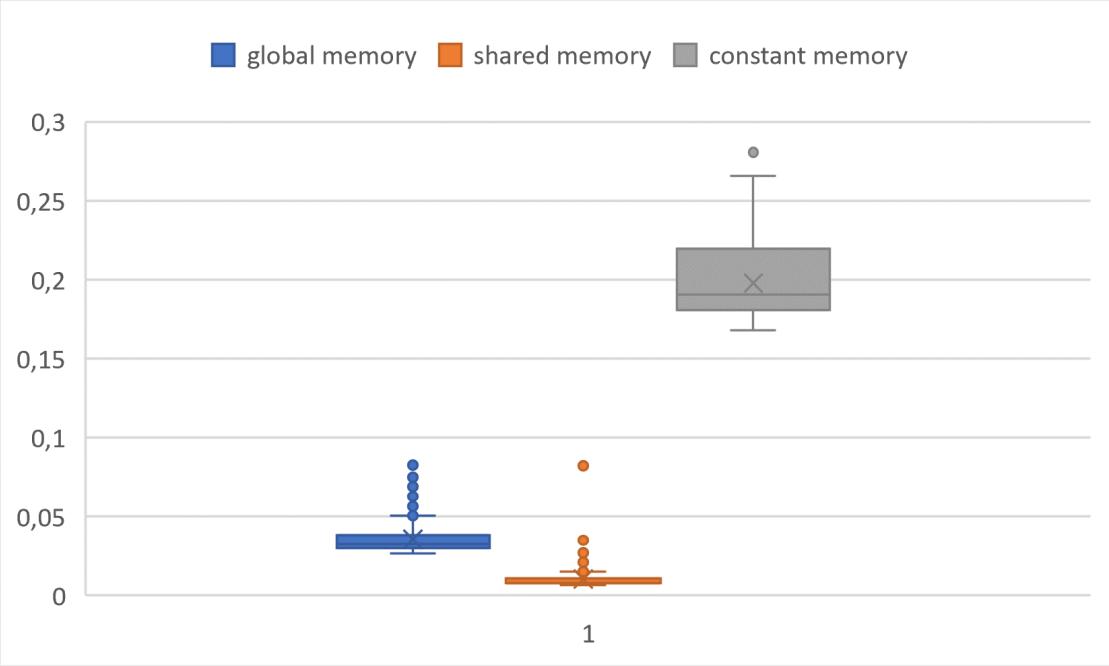


Figure 5: average processing time (ms) (1k runs) for different memory types.

References

[1] "CUDA C++ Programming Guide." https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html, 2024. Accessed: 17 March 2024.